



UNIVERSIDADE ESTADUAL PAULISTA
“JÚLIO DE MESQUITA FILHO”

Compiladores

RELATÓRIO: CONSTRUÇÃO DE UM ANALISADOR SINTÁTICO

Discente: Raí da Silva de Mattos

Discente: Rafaela Granado Castilho

Docente: Prof.^a Dr.^a Renata Spolon Lobato

Docente Assistente: Gabriel Augusto Lima Silva

Instituto de Biociências, Letras e Ciências Exatas – Departamento de Ciências de
Computação e Estatística

São José do Rio Preto – SP – Brasil
Novembro/2025

Sumário

1. INTRODUÇÃO	1
2. GRAMÁTICA LIVRE DE CONTEXTO (GLC)	1
4. MODIFICAÇÕES NO ANALISADOR LÉXICO	3
5. MANUAL DO USUÁRIO	3
5.1 FUNCIONAMENTO	3
5.2. COMPILAÇÃO E EXECUÇÃO	3

1. INTRODUÇÃO

O presente trabalho tem como objetivo implementar um analisador sintático utilizando o gerador Bison, complementado por um analisador léxico construído com Flex. A linguagem analisada foi inspirada em uma variação do Portugol, com comandos escritos em português, de modo a tornar a programação mais acessível a usuários iniciantes. A linguagem inclui declarações de variáveis, comandos de seleção e repetição, atribuições e expressões aritméticas e lógicas, além de suportar funções, constantes e blocos de código.

2. GRAMÁTICA LIVRE DE CONTEXTO (GLC)

A gramática foi definida no arquivo trabSin.y e segue a forma $G = (V, \Sigma, R, S)$, onde:

V (Variáveis / Não-terminais):

```
{programa, lista_comandos, comando, bloco_codigo, comando_if,
comando_while, comando_for, inicializacao_for,
declaracao_completa, declaracao_sem_terminador, tipo,
atribuicao, expressao, termo, fator}
```

Σ (Símbolos Terminais):

```
{TIPO_INT, TIPO_REAL, TIPO_CHAR, TIPO_STRING, VARIABEL,
CONSTANTE, FUNCAO, NUMERO_INT, NUMERO_REAL, CHAR, STRING,
OP_ATRIBUICAO, OP_ARIT_ADD, OP_ARIT_SUB, OP_ARIT_MULT,
OP_ARIT_DIV, OP_LOG_E, OP_LOG_OU, OP_REL_IGUAL,
OP_REL_DIFERENTE, OP_REL_MENOR, OP_REL_MAIOR,
OP_REL_MENOR_IGUAL, OP_REL_MAIOR_IGUAL, KW_IF, KW_ELSE,
```

KW_WHILE, KW_FOR, KW_RETURN, DELIM_ABRE_PAR, DELIM_FECHA_PAR, DELIM_ABRE_CHAVE, DELIM_FECHA_CHAVE, DELIM_PONTOEVIRGULA, DELIM_VIRGULA}

S (Símbolo Inicial): programa

R (Regras de Produção):

1. programa $\rightarrow$ lista_comandos
2. lista_comandos $\rightarrow \epsilon \mid$ lista_comandos comando
3. comando $\rightarrow$ declaracao_completa $\mid$ comando_if $\mid$ comando_while $\mid$ comando_for $\mid$ comando_return $\mid$ bloco_codigo $\mid$ expressao DELIM_PONTOEVIRGULA
4. comando_if $\rightarrow$ KW_IF DELIM_ABRE_PAR expressao DELIM_FECHA_PAR corpo_comando $\mid$ KW_IF DELIM_ABRE_PAR expressao DELIM_FECHA_PAR corpo_comando KW_ELSE corpo_comando
5. comando_while $\rightarrow$ KW_WHILE DELIM_ABRE_PAR expressao DELIM_FECHA_PAR corpo_comando
6. comando_for $\rightarrow$ KW_FOR DELIM_ABRE_PAR inicializacao_for DELIM_PONTOEVIRGULA expressao DELIM_PONTOEVIRGULA expressao DELIM_FECHA_PAR corpo_comando
7. inicializacao_for: declaracao_sem_terminador $\mid$ expressao $\mid \epsilon$
8. declaracao_completa $\rightarrow$ tipo VARIABEL OP_ATRIBUICAO expressao DELIM_PONTOEVIRGULA $\mid$ tipo VARIABEL DELIM_PONTOEVIRGULA
9. bloco_codigo $\rightarrow$ DELIM_ABRE_CHAVE lista_comandos DELIM_FECHA_CHAVE
10. comando_return: KW_RETURN DELIM_PONTOEVIRGULA $\mid$ KW_RETURN expressao DELIM_PONTOEVIRGULA
11. corpo_comando: comando
12. tipo $\rightarrow$ TIPO_INT $\mid$ TIPO_REAL $\mid$ TIPO_CHAR $\mid$ TIPO_STRING
13. expressao $\rightarrow$ expressao OP_ARIT_ADD termo $\mid$ expressao OP_ARIT_SUB termo $\mid$ termo
14. termo $\rightarrow$ termo OP_ARIT_MULTI fator $\mid$ termo OP_ARIT_DIV fator $\mid$ fator
15. fator $\rightarrow$ VARIABEL $\mid$ NUMERO_INT $\mid$ NUMERO_REAL $\mid$ STRING $\mid$ CHAR $\mid$ DELIM_ABRE_PAR expressao DELIM_FECHA_PAR

4. MODIFICAÇÕES NO ANALISADOR LÉXICO

Durante o desenvolvimento, foram feitos alguns ajustes ao trabalho 1, tais como a adição do operador de decremento (--), pois anteriormente existia apenas o incremento (++). A nova regra reconhece o token OP_ARIT_DEC. Também foi feita a mudança da palavra-chave `nao` para `senao`, para evitar ambiguidade com o operador lógico `!`. Agora o `else` é reconhecido como `senao`. O delimitador `/n` foi modificado para `;`, similar ao C e outras linguagens populares de programação.

5. MANUAL DO USUÁRIO

Este manual tem o objetivo de orientar o uso do analisador sintático desenvolvido.

5.1 FUNCIONAMENTO

O analisador sintático verifica a estrutura gramatical dos programas escritos na linguagem proposta, garantindo que todas as instruções sigam a forma definida pela GLC implementada no arquivo `trabSin.y`. Esse componente valida a ordem, combinação e hierarquia dos tokens reconhecidos pelo analisador léxico, assegurando que o código-fonte esteja sintaticamente correto antes de qualquer processamento semântico.

Cada token reconhecido é enviado para o analisador sintático através da função `yylex()`, e o Bison compara os tokens recebidos com as regras da gramática. Se os tokens correspondem às regras previstas, a análise prossegue, caso contrário, é adicionada a função de erro `yyerror()`. O Bison implementa análise LR(1), construindo uma derivação mais à direita da gramática. Ou seja, o token aceito pelo analisador léxico é empilhado, e há a tentativa de redução conforme as regras da gramática. Caso uma regra seja reconhecida, o analisador sintático substitui os símbolos da pilha pelo não terminal correspondente, o que chamamos de redução. Esse processo se repete até que o programa inicial seja aceito, ou algum erro seja detectado.

5.2. COMPILAÇÃO E EXECUÇÃO

Passos para compilação:

```
flex lexicoTrabalho.l
```

```
bison -d trabSin.y
```

```
gcc lex.yy.c trabSin.tab.c -o meu_analisador
```

... e para execução:

```
./meu_analisador < teste.txt
```

Saídas esperadas:

Entrada correta: Análise Sintática: SUCESSO

Entrada incorreta:

Análise Sintática: FALHA

--- ERRO SINTÁTICO ---

Linha: (número da linha)

Trecho encontrado: (print da linha)

Detalhe: syntax error, unexpected (token reconhecido pelo analisador léxico, porém inesperado pelo sintático), expecting (token esperado pelo analisador sintático)
